

SECTION 3 — ADVANCED JAVA CONCEPTS (Senior SDET Masterclass)

Topics Covered

- 3.1 `equals()` / `hashCode()` Contract (The Low-Level Mechanics of Breaking Collections)
 - 3.2 `toString()` (Logging, Failure Triage, & Memory Footprint)
 - 3.3 Mutable Objects as Map Keys (Silent Lookup Failures & Memory Leaks)
 - 3.4 String Immutability (Security, Thread Safety, & JVM Optimization)
 - 3.5 String Constant Pool (Interning, Metaspace/Heap Migration, & Compaction)
 - 3.6 `==` vs `equals()` (Identity vs Structural Equality Traps)
 - 3.7 `String` vs `StringBuilder` vs `StringBuffer` (Buffer Growth & Thread Contention)
 - 3.8 `final` vs Immutable (Reference Immutability vs State Immutability)
 - 3.9 Wrapper Classes (Caching Mechanics, Memory Inflation, & Nullability)
 - 3.10 Autoboxing / Unboxing (Performance Degradation & NPE Traps)
 - 3.11 `enum` (Type-Safe Singletons, Strategy Enums, & Test Configuration)
 - 3.12 Inner Classes (Member Classes & Outer Instance References)
 - 3.13 Anonymous Classes (Bytecode Generation & Lambda Contrast)
 - 3.14 Nested Static Classes (Decoupled Namespace Organization)
 - 3.15 Varargs (Heap Pollution, Array Allocation, & Overload Ambiguities)
 - 3.16 Annotations (Retention Policies, Reflection Metadata, & Custom Test Annotations)
 - 3.17 Reflection API (Dynamic Proxying, Framework Extensibility, & Security Risks)
 - 3.18 Serialization & Deserialization (`serialVersionUID` , Security Vulnerabilities, & POJOs)
 - 3.19 `transient` Keyword (Excluding Secrets & Non-Serializable Resources)
 - 3.20 `volatile` Keyword (Visibility, Reordering, & Memory Barriers)
 - 3.21 `synchronized` Keyword (Monitors, Lock Biasing, & Concurrency Contention)
 - 3.22 `Atomic` Classes (CAS Mechanics & Lock-Free Performance)
 - 3.23 Immutability & Thread Safety (Safe Publication & Memory Consistency)
 - 3.24 High-Stakes Advanced Java Interview Questions & Spoken Solutions
-

3.1 `equals()` / `hashCode()` Contract

1. Theory & Core Mechanics

The contract governing `Object.equals(Object)` and `Object.hashCode()` dictates how hash-based collections (`HashMap` , `HashSet` , `Hashtable` , `ConcurrentHashMap`) locate and store entries:

1. **Reflexive:** `x.equals(x)` must be `true` .
2. **Symmetric:** `x.equals(y)` returns `true` if and only if `y.equals(x)` returns `true` .
3. **Transitive:** If `x.equals(y)` is `true` and `y.equals(z)` is `true` , then `x.equals(z)` must be `true` .
4. **Consistent:** Multiple invocations must return the same result unless state is mutated.
5. **Non-nullity:** `x.equals(null)` must always return `false` .

6. The Fundamental Hash Invariant: If `x.equals(y)` is `true`, then `x.hashCode() == y.hashCode()` **MUST** be true.

(Note: The reverse is not required: `x.hashCode() == y.hashCode()` does NOT require `x.equals(y)` to be true—this is merely a hash collision).

When searching `HashMap.get(key)`:

1. Compute `hash = hash(key.hashCode())`
2. Determine bucket index = `(capacity - 1) & hash`
3. Traverse bucket linked list / tree node:
Match if: `(node.hash == hash) && (node.key == key || key.equals(node.key))`

If you override `equals()` but fail to override `hashCode()`, two identical objects produce different hash codes. They map to completely different buckets, causing `map.get(key)` to return `null` even though the key conceptually exists.

2. Enterprise Relevance (5,000+ Test Scale)

In test automation, custom DTOs represent test users, orders, or API responses. If these DTOs are stored in a `HashSet` to dedup test scenarios or used as keys in a `Map<UserDTO, WebDriver>`, breaking this contract leads to:

- Duplicate test executions running in parallel on what was thought to be deduplicated data.
- Cache misses in token and session managers, causing tests to flood identity providers with redundant OAuth requests.

3. Production-Ready Code: Robust Contract Implementation

```
package com.deloitte.sdet.model;

import java.util.Objects;

public final class TestUserAccount {

    private final String tenantId;
    private final String userEmail;
    private final int accessLevel;

    public TestUserAccount(String tenantId, String userEmail, int accessLevel) {
        this.tenantId = Objects.requireNonNull(tenantId, "tenantId cannot be null");
        this.userEmail = Objects.requireNonNull(userEmail, "userEmail cannot be null");
        this.accessLevel = accessLevel;
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) return true; // Reference identity optimization
        if (!(obj instanceof TestUserAccount other)) return false; // Null-safe pattern matching
        return this.accessLevel == other.accessLevel
            && Objects.equals(this.tenantId, other.tenantId)
            && Objects.equals(this.userEmail, other.userEmail);
    }
}
```

```

@Override
public int hashCode() {
    // Must utilize EXACT SAME fields used in equals()
    return Objects.hash(tenantId, userEmail, accessLevel);
}

public String getTenantId() { return tenantId; }
public String getUserEmail() { return userEmail; }
public int getAccessLevel() { return accessLevel; }
}

```

4. High-Stakes Scenario: Duplicate Execution Flooding CI

- **Context:** A test runner reads 2,000 test cases into a `Set<TestCaseDescriptor>` to eliminate duplicate regression runs. The runner still executes 2,000 tests instead of the expected 1,200 unique tests.
- **Triage:**
 1. Inspect `TestCaseDescriptor` : Found an overridden `equals()` method checking `testId` and `methodName`, but `hashCode()` was omitted (relying on default `System.identityHashCode()`).
 2. Every instance created by deserializing JSON files received a distinct memory address, generating unique hash codes.
 3. Resolution: Implement `Objects.hash(testId, methodName)` or convert the class to a Java `record` `TestCaseDescriptor(String testId, String methodName) {}`.

3.2 toString()

1. Theory & Core Mechanics

The default implementation in `java.lang.Object` returns `getClass().getName() + "@" + Integer.toHexString(hashCode())`. This provides zero visibility into the internal state of an object.

- A custom `toString()` should provide a deterministic, human-readable summary of state for logging and assertions.
- **Security Constraint:** Must never print sensitive credentials, tokens, or PII.

2. Production-Ready Code: Safe `toString()` with Masking

```

package com.deloitte.sdet.model;

public record ApiCredential(String clientId, String clientSecret, String tokenEndpoint) {

    @Override
    public String toString() {
        // Strict security masking for test reporting and logs
        String maskedSecret = (clientSecret != null && clientSecret.length() > 4)
            ? "*****" + clientSecret.substring(clientSecret.length() - 4)
            : "*****";
        return String.format("ApiCredential[clientId='%s', clientSecret='%s', tokenEndpoint='%s'"
            + "clientId, maskedSecret, tokenEndpoint);

```

```
}
}
```

3.3 Mutable Objects as Map Keys

1. Theory & Silent Lookup Failures

When an object is placed into a `HashMap`, its bucket index is computed from its current `hashCode()`. If that object's fields are subsequently mutated such that its `hashCode()` changes:

1. The object remains physically sitting in the **old bucket**.
2. Calling `map.get(key)` recalculates the hash based on the **new mutated values**, pointing to a **different bucket**.
3. Result: `map.get(key)` returns `null`!
4. The entry can no longer be retrieved or removed via standard keys, creating a **silent memory leak**.

```
// SEVERE MEMORY LEAK & LOOKUP FAILURE DEMO
Map<List<String>, String> permissions = new HashMap<>();
List<String> roles = new ArrayList<>(List.of("READ"));

permissions.put(roles, "STANDARD_USER");

// Mutate after storing in map
roles.add("WRITE");

System.out.println(permissions.get(roles)); // Prints null!
System.out.println(permissions.containsKey(roles)); // Prints false!
System.out.println(permissions.size()); // Prints 1 (entry is stranded in memory)
```

Architectural Rule: Map keys must always be immutable. Use `String`, `Integer`, `UUID`, or immutable records.

3.4 String Immutability

1. Theory & Mechanics

In Java, `java.lang.String` is completely immutable:

- The class is declared `final`.
- The internal character buffer (`byte[]` value in Java 9+ compact strings) is `private final`.
- No mutator methods exist (`replace()`, `toLowerCase()` create and return a brand-new `String` instance).

Why Strings are Immutable in Java:

1. String Pool	Allows thousands of references to share 1 entry
2. Thread Safety	Read-only; zero synchronization required
3. Security	File paths, URLs, DB creds cannot be mutated

4. Hash Caching	Hash code is computed once and cached lazily
-----------------	--

3.5 String Constant Pool (SCP)

1. Theory & Memory Relocation

The String Constant Pool is a special storage area managed by the JVM:

- Prior to Java 7: Stored in **PermGen** (fixed size, prone to `OutOfMemoryError: PermGen space`).
- Java 7+: Moved to the standard **Java Heap**, allowing unused interned strings to be garbage collected.

String Allocation Mechanics

- String literals (`String s = "hello"`) check the SCP. If present, returns reference; if absent, creates in SCP.
- String object creation (`String s = new String("hello")`) creates **two objects**: one in SCP (if not present) and one explicit object on the heap.
- `s.intern()` : Forces the string into the SCP and returns the canonical pool reference.

```
String s1 = "deloitte";
String s2 = "deloitte";
String s3 = new String("deloitte");
String s4 = s3.intern();

System.out.println(s1 == s2); // true (Identical SCP reference)
System.out.println(s1 == s3); // false (s3 is a separate Heap object)
System.out.println(s1 == s4); // true (s4 points to the SCP reference)
```

3.6 == VS equals()

Comparison	Operands	What it Evaluates	Example
== Operator	Primitives	Direct bit-value equality.	<code>5 == 5</code> \$to\$ <code>true</code> .
== Operator	Objects	Reference Identity : Whether both pointers reference the exact same memory address on the heap.	<code>new String("a") == new String("a")</code> \$to\$ <code>false</code> .
.equals() Method	Objects	Structural Content Equality : Overridden to compare internal state and logical values.	<code>"a".equals(new String("a"))</code> \$to\$ <code>true</code> .

3.7 String VS StringBuilder VS StringBuffer

Class	Mutability	Thread Safety	Performance & Use Case
-------	------------	---------------	------------------------

`String`	Immutable	Thread-Safe	Constants, map keys, DTOs
`StringBuilder`	Mutable	NOT Safe	Fast single-threaded work
`StringBuffer`	Mutable	Thread-Safe	Legacy synchronized loops

The String Concatenation Loop Trap

```
// CATASTROPHIC ANTI-PATTERN: O(N²) time complexity and thousands of heap allocations
String report = "";
for (int i = 0; i < 10_000; i++) {
    report += "TestResult-" + i + "\n"; // Allocates new StringBuilder & String every pass!
}

// OPTIMAL SENIOR PATTERN: O(N) time with pre-sized buffer
StringBuilder builder = new StringBuilder(10_000 * 20);
for (int i = 0; i < 10_000; i++) {
    builder.append("TestResult-").append(i).append("\n");
}
String finalReport = builder.toString();
```

3.8 final vs. Immutable

- **final Reference:** Prevents reassigning the variable pointer to another memory location. **It does NOT prevent mutating the object itself!**

```
final List<String> browsers = new ArrayList<>();
browsers.add("Chrome"); // Perfectly legal! Internal state mutates.
// browsers = new ArrayList<>(); // COMPILER ERROR: Cannot reassign final pointer
```

- **Immutable Object:** The internal state of the object cannot change after creation (`List.of("Chrome")`).

3.9 Wrapper Classes & Caching Mechanics

The 8 primitives have corresponding object wrappers (`Integer` , `Double` , `Boolean` , etc.).

The Integer Cache Trapping Senior Engineers

The JVM caches `Integer` instances within the range **-128 to 127** (via `Integer.IntegerCache`):

```
Integer a = 127;
Integer b = 127;
System.out.println(a == b); // true (Pulls from IntegerCache)

Integer c = 128;
Integer d = 128;
```

```
System.out.println(c == d); // FALSE! (Allocates new separate Heap objects)
System.out.println(c.equals(d)); // true (Compares unboxed int values)
```

Golden Rule: Never compare wrapper objects using `==`; always use `.equals()`.

3.10 Autoboxing and Unboxing

- **Autoboxing:** Automatic conversion of primitives to their wrapper object (`int` $\rightarrow$ `Integer`).
- **Unboxing:** Automatic conversion of wrapper objects to primitives (`Integer` $\rightarrow$ `int`).

Dangerous Gotcha: NullPointerExceptions on Unboxing

```
public class UnboxingTrap {
    public static void validateStatusCode(int statusCode) {
        System.out.println("Status: " + statusCode);
    }

    public static void main(String[] args) {
        Integer cachedCode = null;
        // Throws java.lang.NullPointerException at runtime:
        // JVM invokes cachedCode.intValue() behind the scenes!
        validateStatusCode(cachedCode);
    }
}
```

3.11 `enum` Mechanics in Framework Design

1. Theory & Core Mechanics

In Java, an `enum` is a specialized class extending `java.lang.Enum<E>`.

- Inherently serializable and immutable.
- Constructor is implicitly `private`.
- Provides a guaranteed **Thread-Safe Singleton** pattern (JVM guarantees single instance per constant).

2. Production Code: Strategy Enum for Test Execution

```
package com.deloitte.sdet.enums;

import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;
import org.openqa.selenium.firefox.FirefoxDriver;

public enum BrowserEnvironment {
    CHROME {
        @Override
        public WebDriver createDriver() {
            return new ChromeDriver();
        }
    }
}
```

```

    },
    FIREFOX {
        @Override
        public WebDriver createDriver() {
            return new FirefoxDriver();
        }
    };

    public abstract WebDriver createDriver();
}

```

3.12 Inner, Anonymous, and Nested Classes

JAVA NESTED CLASS TAXONOMY		
Class Type	Static Context	Enclosing Instance Access
Static Nested Class	`static class`	NO reference to outer `this`
Member Inner Class	Non-static class	Implicit ref to outer `this`
Local Inner Class	Defined in method	Captures effectively-final
Anonymous Class	Ad-hoc inline	Captures effectively-final

[!WARNING]

Memory Leak Risk with Non-Static Inner Classes: Non-static inner classes retain an invisible implicit reference to their enclosing outer class (`Outer.this`). If an inner class instance is retained (e.g., in a listener or thread pool), the entire outer class cannot be garbage collected!

3.13 Annotations & Reflection API

1. Retention Policies

- **SOURCE** : Discarded during compilation (e.g. `@Override` , `@SuppressWarnings`).
- **CLASS** : Recorded in `.class` file, but discarded at runtime (default).
- **RUNTIME** : Retained in Metaspace and accessible via Reflection (`Class.getAnnotation()`). Mandatory for test runners!

2. Production Code: Custom SDET Retry & Issue Tracking Annotation

```

package com.deloitte.sdet.annotations;

import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

```

```
import java.lang.annotation.Target;

@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.METHOD, ElementType.TYPE})
public @interface JiraDefect {
    String issueKey();
    int maxRetries() default 2;
    boolean autoQuarantine() default false;
}
```

3. Reflection in Test Listeners

```
package com.deloitte.sdet.listeners;

import com.deloitte.sdet.annotations.JiraDefect;
import java.lang.reflect.Method;

public final class AnnotationInspector {

    public static void inspectTestExecution(Class<?> testClass, String methodName) {
        try {
            Method method = testClass.getMethod(methodName);
            if (method.isAnnotationPresent(JiraDefect.class)) {
                JiraDefect defect = method.getAnnotation(JiraDefect.class);
                System.out.printf("Test [%s] linked to Jira: %s | Max Retries: %d\n",
                    methodName, defect.issueKey(), defect.maxRetries());
            }
        } catch (NoSuchMethodException e) {
            throw new RuntimeException("Method not found for reflection inspection", e);
        }
    }
}
```

3.14 Serialization, Deserialization, & `transient`

- **Serialization:** Converting in-memory object graphs into a portable byte stream.
- **`serialVersionUID`** : Unique version identifier for a `Serializable` class. If omitted, the JVM calculates one using class hashing. Any change to class structure alters the UID, throwing `InvalidClassException` on deserialization.
- **`transient` Keyword:** Instructs the JVM serialization mechanism to skip this field. The field is restored to its default value (`null` , `0`) upon deserialization. Essential for sensitive tokens and un-serializable objects (like `WebDriver`).

3.15 Concurrency Fundamentals: `volatile` , `synchronized` , and `Atomic`

CONCURRENCY MECHANISMS

<code>`volatile`</code>	Visibility & Order: Reads/writes bypass CPU L1/L2 caches directly to main memory. No mutual exclusion.
<code>`synchronized`</code>	Mutual Exclusion & Visibility: Acquires monitor lock. Guarantees atomicity for multi-step operations.
<code>`Atomic*`</code>	Lock-Free Atomicity: Uses CPU hardware CAS (Compare-And-Swap) instructions for peak throughput.

1. `volatile` (Visibility Without Atomicity)

`volatile` guarantees that any thread reading the field sees the most recent write made by any other thread. It prevents CPU instruction reordering via **Memory Barriers**.

Limitation: `volatile int count; count++;` is NOT thread-safe! `count++` consists of 3 distinct bytecode operations: Read `$to$` Increment `$to$` Write.

2. `AtomicInteger` (Lock-Free Thread-Safe Counters)

```
package com.deloitte.sdet.core;

import java.util.concurrent.atomic.AtomicInteger;

public final class ParallelExecutionTracker {
    private final AtomicInteger passedTests = new AtomicInteger(0);
    private final AtomicInteger failedTests = new AtomicInteger(0);

    public void recordPass() { passedTests.incrementAndGet(); }
    public void recordFail() { failedTests.incrementAndGet(); }

    public int getPassCount() { return passedTests.get(); }
    public int getFailCount() { return failedTests.get(); }
}
```

3.16 High-Stakes Senior Java Interview Questions & Spoken Solutions

Q1: "Why does `hashCode()` use the prime number 31 in standard IDE generators and JDK classes?"

"The number 31 is chosen for two distinct engineering reasons:

- 1. It is an odd prime. Multiplying by an even number (like 2) shifts bits to the left, losing information on arithmetic overflow and causing hash collisions. Multiplying by a prime produces a more uniform hash distribution across hash table buckets.*
- 2. Modern JVM JIT compilers automatically optimize multiplication by 31 into an ultra-fast bit shift and subtraction on the CPU: $31 * i == (i \ll 5) - i$. This reduces computation time down to single CPU cycles."*

Q2: "Can `volatile` replace `synchronized` when synchronizing access to a shared `WebDriver` instance?"

"No, absolutely not. `volatile` only guarantees **visibility** (ensuring all threads read the latest reference from main memory) and prevents instruction reordering. It provides zero **mutual exclusion (atomicity)**.*

WebDriver is inherently non-thread-safe. Interacting with a browser (e.g., navigating, finding an element, clicking) involves multi-step HTTP protocol handshakes. If two parallel threads invoke actions on a `volatile WebDriver`, their commands interleave over the socket, causing `NoSuchSessionException` or browser crashes.

*For parallel testing, the correct architectural solution is **thread confinement** using `ThreadLocal<WebDriver>`, which guarantees that each executing thread operates on its own dedicated driver instance with zero lock contention."*

3.25 String Architecture & JVM Memory Layout Masterclass

Java 9+ Compact Strings (`byte[]` vs `char[]`)

Prior to Java 9, `String` stored characters in a `char[]` array where every character consumed 2 bytes (16 bits) using UTF-16 encoding. Since over 85% of heap-allocated strings in typical enterprise applications consist only of single-byte ISO-8859-1 / Latin-1 characters, Java 9 introduced **Compact Strings**:

```
// Java 9+ String Internal Declaration
public final class String implements java.io.Serializable, Comparable<String>, CharSequence {
    @Stable
    private final byte[] value; // Replaced char[] value
    private final byte coder;   // 0 = LATIN1 (1 byte/char), 1 = UTF16 (2 bytes/char)
    private int hash;           // Cached hash code (defaults to 0)
}
```

Exhaustive Java `String` Method Master Reference Table

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>length()</code>	<code>int</code>	Returns number of UTF-16 code units.	$O(1)$
<code>isEmpty()</code>	<code>boolean</code>	Returns <code>true</code> if <code>length() == 0</code> .	$O(1)$
<code>isBlank()</code> (Java 11+)	<code>boolean</code>	Returns <code>true</code> if empty or contains only whitespace.	$O(N)$

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>charAt(int index)</code>	<code>char</code>	Returns <code>char</code> value at specified index.	$O(1)$
<code>codePointAt(int index)</code>	<code>int</code>	Returns Unicode code point at index.	$O(1)$
<code>substring(int beginIndex)</code>	<code>String</code>	Returns substring from <code>beginIndex</code> to end.	$O(N)$
<code>substring(int begin, int end)</code>	<code>String</code>	Returns substring from <code>begin</code> to <code>end - 1</code> .	$O(N)$
<code>indexOf(String str)</code>	<code>int</code>	Returns index of first occurrence of substring, or -1.	$O(N \times M)$
<code>lastIndexOf(String str)</code>	<code>int</code>	Returns index of last occurrence of substring.	$O(N \times M)$
<code>contains(CharSequence s)</code>	<code>boolean</code>	Checks if string contains sequence.	$O(N \times M)$
<code>startsWith(String prefix)</code>	<code>boolean</code>	Checks if string starts with prefix.	$O(K)$
<code>endsWith(String suffix)</code>	<code>boolean</code>	Checks if string ends with suffix.	$O(K)$
<code>equals(Object anObject)</code>	<code>boolean</code>	Structural character-by-character equality check.	$O(N)$
<code>equalsIgnoreCase(String s)</code>	<code>boolean</code>	Case-insensitive structural equality check.	$O(N)$
<code>compareTo(String s)</code>	<code>int</code>	Lexicographical comparison (<0 , 0 , >0).	$O(N)$
<code>toLowerCase()</code>	<code>String</code>	Converts all characters to lowercase.	$O(N)$
<code>toUpperCase()</code>	<code>String</code>	Converts all characters to uppercase.	$O(N)$
<code>trim()</code>	<code>String</code>	Removes leading & trailing characters $\text{\u0020}$.	$O(N)$
<code>strip()</code> (Java 11+)	<code>String</code>	Removes all Unicode leading & trailing whitespace.	$O(N)$
<code>stripLeading()</code> (Java 11+)	<code>String</code>	Removes leading Unicode whitespace.	$O(N)$

Method Signature	Return Type	Description & SDET Use Case	Time Complexity
<code>stripTrailing()</code> (Java 11+)	<code>String</code>	Removes trailing Unicode whitespace.	$O(N)$
<code>repeat(int count)</code> (Java 11+)	<code>String</code>	Repeats string <code>count</code> times.	$O(N \times \text{count})$
<code>replace(CharSequence target, CharSequence replacement)</code>	<code>String</code>	Replaces all literal target occurrences.	$O(N)$
<code>replaceAll(String regex, String replacement)</code>	<code>String</code>	Replaces regex matches using Pattern matcher.	$O(N)$
<code>replaceFirst(String regex, String replacement)</code>	<code>String</code>	Replaces first regex match.	$O(N)$
<code>split(String regex)</code>	<code>String[]</code>	Splits string around regex matches.	$O(N)$
<code>split(String regex, int limit)</code>	<code>String[]</code>	Splits string around regex matches up to <code>limit</code> .	$O(N)$
<code>join(CharSequence delimiter, CharSequence... elements)</code>	<code>String</code>	Joins elements with delimiter.	$O(N)$
<code>matches(String regex)</code>	<code>boolean</code>	Compares string against full regex pattern.	$O(N)$
<code>toCharArray()</code>	<code>char[]</code>	Converts string to new character array.	$O(N)$
<code>getBytes(Charset charset)</code>	<code>byte[]</code>	Encodes string into byte array.	$O(N)$
<code>intern()</code>	<code>String</code>	Returns canonical representation from SCP.	$O(1)$ avg
<code>formatted(Object... args)</code> (Java 15+)	<code>String</code>	Instance method equivalent of <code>String.format()</code> .	$O(N)$

String vs StringBuilder vs StringBuffer Comparison

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread Safety	Thread-Safe	Not Thread-Safe	Thread-Safe (Synchronize)
Performance	Slow for concatenation	Fastest ($O(N)$ buffer)	Slower due to lock overhead
Storage Location	Heap / SCP	Heap	Heap
Buffer Growth Math	N/A	$(oldCapacity * 2) + 2$	$(oldCapacity * 2) + 2$